

Part 7: Java Thread Pool & ForkJoinPool

Custom thread pool:

```
ThreadPoolExecutor executor = newThreadPoolExecutor(  
    2, 2, 0L, TimeUnit.MILLISECONDS,  
    newLinkedBlockingQueue<>(),  
    Executors.defaultThreadFactory(),  
    newThreadPoolExecutor.AbortPolicy()  
);
```

Trong `java.util.concurrent`, có 4 loại pool phổ biến: Fixed, Cached, Single Thread, và WorkStealing (Fork-Join).

1. Fixed Thread Pool

```
public static ExecutorService newFixedThreadPool(int nThreads) {  
    return newThreadPoolExecutor(nThreads, nThreads,           //core = max = nThreads  
                                0L, TimeUnit.MILLISECONDS,    //keepAliveTime = 0  
                                newLinkedBlockingQueue<Runnable>()); // queue không giới hạn  
}
```

- tạo pool với số thread cố định
- Trường hợp sử dụng: biết trước chính xác số luồng, số luồng hệ thống cần
- Nhược điểm: workload → queue phình to → tăng độ trễ

```

public static ExecutorService newFixedThreadPool(int nThreads, ThreadFactory
threadFactory) {
    return new ThreadPoolExecutor(nThreads, nThreads,
                                0L, TimeUnit.MILLISECONDS,
                                new LinkedBlockingQueue<Runnable>(),
                                threadFactory); //truyền vào một Factory
}

```

2. Cached Thread Pool

```

public static ExecutorService newCachedThreadPool() {
    return new ThreadPoolExecutor(0,           //core = 0
                                Integer.MAX_VALUE, // max = Integer.MAX_VALUE
                                60L, TimeUnit.SECONDS, //keepAliveTime = 60s
                                new SynchronousQueue<Runnable>()); //blocking queue với size = 0
}

```

- tạo thread mới linh hoạt khi cần và hủy khi rảnh quá lâu
- trường hợp sử dụng: Workload dạng burst - nhiều task ngắn hạn, số lượng không đoán trước, cần xử lý song song tối đa
- Nhược điểm: quá nhiều task nặng đưa vào pool → tạo nhiều thread cùng lúc → tăng mức sử dụng bộ nhớ

3. Single Thread Executor

```

public static ExecutorService newSingleThreadExecutor() {
    return newSingleThreadExecutor(defaultThreadFactory());
}

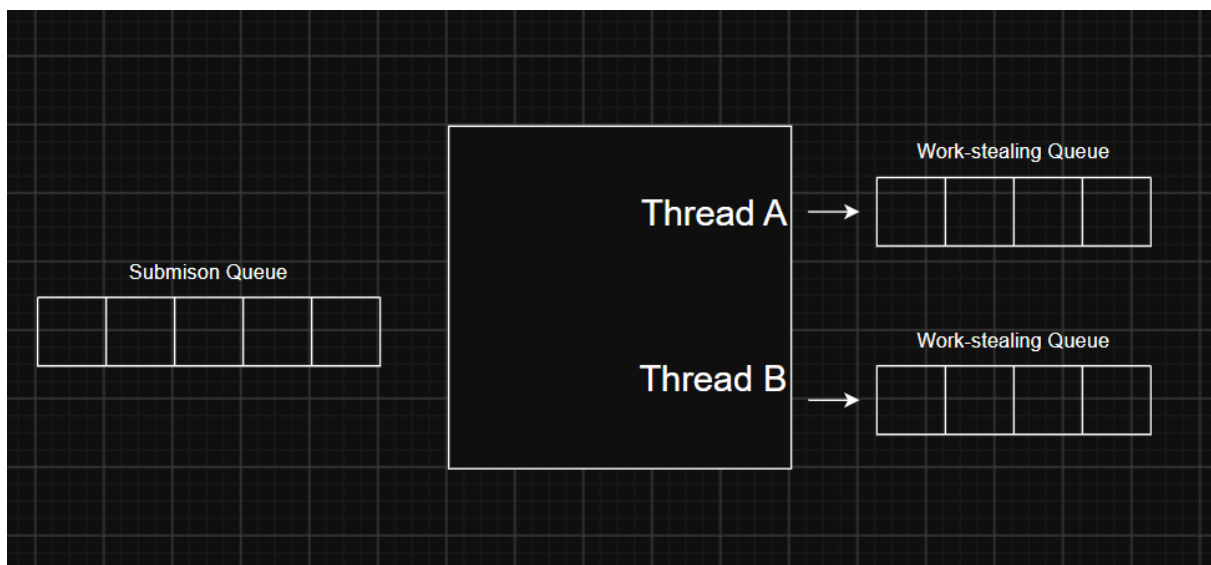
public static ExecutorService newSingleThreadExecutor(ThreadFactory threadFactory) {
    return new AutoShutdownDelegatedExecutorService
        (new ThreadPoolExecutor(1, 1,                // core = max = 1
                                0L, TimeUnit.MILLISECONDS, // keepAliveTime = 0
                                new LinkedBlockingQueue<Runnable>(), // queue không giới hạn
                                threadFactory));           // factory
}

```

- Chỉ có duy nhất 1 worker thread làm việc
- Trường hợp sử dụng: cần xử lý các tác vụ tuần tự, đúng thứ tự submit
- Nhược điểm: hoàn toàn không có concurrency.

4. WorkStealing Pool (Fork-Join)

- Tạo 1 **ForkJoinPool**
- Số lượng thread mặc định = số nhân CPU (`availableProcessors()`), có thể tự chỉ định qua tham số.



- 2 loại hàng đợi:
 - **Submission Queue:** hàng đợi chung của cả Pool.

- **Work-Stealing Queue:** mỗi thread có **1 Deque riêng**
- **Cơ chế Steal:**
 - Thread A nhận task 1 từ Submission Queue, gọi fork() tách thành Subtask 1 và Subtask 2
 - Subtask 1: Thread A lấy và thực thi ngay
 - Subtask 2: nằm ở deque riêng của A
 - Nếu thread B rảnh và Submission Queue trống, B sẽ steal Subtask 2 từ deque A (theo LIFO) để xử lý giúp → không thread nào rảnh trong khi thread khác quá tải, tối ưu hiệu suất.
- Task muốn chia nhỏ theo cơ chế này phải kế thừa:
 - RecursiveTask<V> : có kiểu trả về
 - RecursiveAction: không có kiểu trả về
- **2 cách khởi tạo:**
 - Executors.newWorkStealingPool([n]): tạo pool **riêng**
 - ForkJoinPool.commonPool(): **dùng chung cho cả JVM**